Concurrent Programming: Languages and techniques

Presentation of Rust

António Ravara

(with Bernardo Toninho and Carla Ferreira)

Departament of Computer Science,
NOVA School of Science and Technology

Motivating Rust

The Rust Language

Purpose

A systems programming language that is *safe* and *fast*.

The Rust Language

Purpose

A systems programming language that is *safe* and *fast*.

Systems programming?

Meant for writing OSs or device drivers, not applications (but one can, of course!).

The Rust Language

Purpose

A systems programming language that is *safe* and *fast*.

Systems programming?

Meant for writing OSs or device drivers, not applications (but one can, of course!).

'*Fast*' is easy to understand, but

What do they mean by *safe*?

That's the aim of this lecture...

Motivation

C/C++: neither safe nor precise

Why a new language? Aren't C or C++ great?

Yes, but...

Impreciseness:*undefined* and/or *unspecified* behaviour

According to the manuals, some instructions, in certain conditions, are compiler dependent

C/C++: neither safe nor precise

Why a new language? Aren't C or C++ great?

Yes, but...

Impreciseness:*undefined* and/or *unspecified* behaviour

According to the manuals, some instructions, in certain conditions, are compiler dependent

It is very challenging to get programs right

Bugs are really frequent... Source of problems:

- memory management;
- concurrency (data-races).

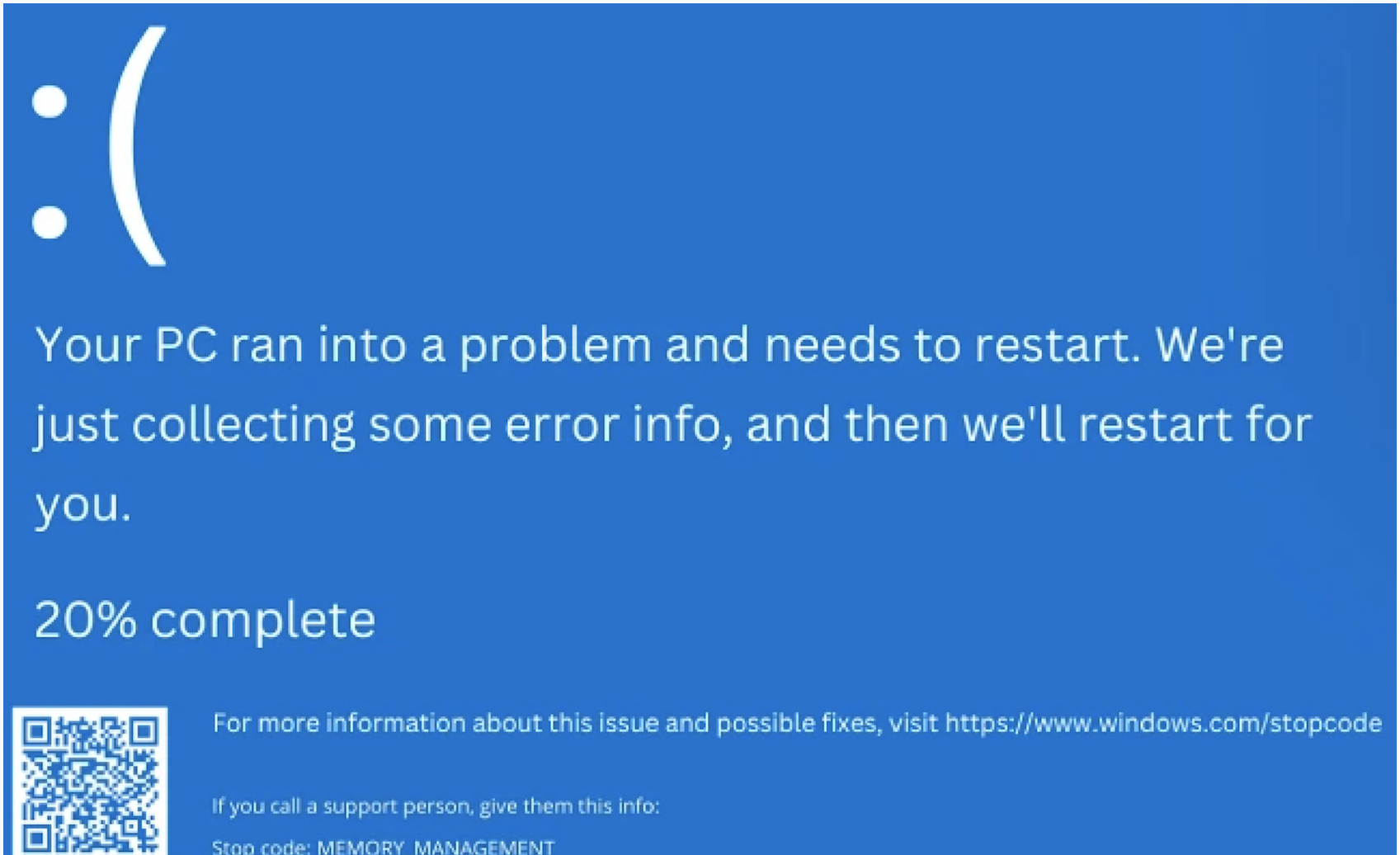
Help: external static analyses tools

`https://en.wikipedia.org/wiki/List_of_tools_for_static_code_analysis`

Memory unsafety

Matt Miller, MS security engineer, 2019

> "Around 70% of all vulnerabilities in MS products, addressed through a security update each year, are memory safety issues"



Memory unsafety

Null reference – the billion dollar mistake, 2009

> "It was the invention of the null reference in 1965... This has led to innumerable errors, vulnerabilities, and system crashes, which have probably caused a billion dollars of pain and damage in the last forty years."

C.A.R. Hoare, creator of Algol and QuickSort; winner of the Turing Prize (the Nobel Prize in Computing)

Memory unsafety in C/C++

Seg-faults

- use-after-free
- double free
- buffer overreads/overwrites
- null pointer de-refs

Memory unsafety in C/C++

Seg-faults

- use-after-free
- double free
- buffer overreads/overwrites
- null pointer de-refs

And that's not all:

- memory leaks (dangling pointers)
- data-races

Examples of memory-related bugs in C

Consider the code snippet (with `l` a linked list)

```
l = l -> next;
```

Used-after free: `free(l)` might have been happened before

Examples of memory-related bugs in C

Consider the code snippet (with `l` a linked list)

```
l = l -> next;
```

Used-after free: `free(l)` might have been happened before

Null de-ref: `printf(l -> val)` might also fail if we have reached the end of the list...

Examples of memory-related bugs in C

Consider the code snippet (with `l` a linked list)

```
l = l -> next;
```

Used-after free: `free(l)` might have been happened before

Null de-ref: `printf(l -> val)` might also fail if we have reached the end of the list...

Double free: `free(l)` can crash if the memory address was already freed

Examples of memory-related bugs in C

Consider the code snippet (with `l` a linked list)

```
l = l -> next;
```

Used-after free: `free(l)` might have been happened before

Null de-ref: `printf(l -> val)` might also fail if we have reached the end of the list...

Double free: `free(l)` can crash if the memory address was already freed

Other seg. faults: buffer overflows

Examples of memory-related bugs in C

Consider the code snippet (with `l` a linked list)

```
l = l -> next;
```

Used-after free: `free(l)` might have been happened before

Null de-ref: `printf(l -> val)` might also fail if we have reached the end of the list...

Double free: `free(l)` can crash if the memory address was already freed

Other seg. faults: buffer overflows

Memory leaks: unreleased memory no longer accessible (like addresses stored in the cells from `next` not freed)

A more elaborate example, in C++

Iterator invalidator

```
std::vector<int> v { 10, 11 };//Declares&initialises
int *vptr = &v[1];//Points *into* ‘v’
v.push_back(12);//Inserts value 12
std::cout « *vptr;
```

A more elaborate example, in C++

Iterator invalidator

```
std::vector<int> v { 10, 11 };//Declares&initialises
int *vptr = &v[1];//Points *into* ‘v’
v.push_back(12);//Inserts value 12
std::cout « *vptr;//Bug (possible use-after-free)
```

A more elaborate example, in C++

Iterator invalidator

```
std::vector<int> v { 10, 11 };//Declares&initialises
int *vptr = &v[1];//Points *into* 'v'
v.push_back(12);//Inserts value 12
std::cout « *vptr;//Bug (possible use-after-free)
```

Bug?

`push_back` can allocate a new vector (if `v` full)
then, the reference `vptr` points to an invalid memory location
(it's a "dangling pointer") – `*vptr` might go wrong

A more elaborate example, in C++

Iterator invalidator

```
std::vector<int> v { 10, 11 };//Declares&initialises
int *vptr = &v[1];//Points *into* 'v'
v.push_back(12);//Inserts value 12
std::cout « *vptr;//Bug (possible use-after-free)
```

Bug?

`push_back` can allocate a new vector (if `v` full)
then, the reference `vptr` points to an invalid memory location
(it's a "dangling pointer") – `*vptr` might go wrong

We're humans: "42 little bugs in my code; 42 little bugs;

A more elaborate example, in C++

Iterator invalidator

```
std::vector<int> v { 10, 11 };//Declares&initialises
int *vptr = &v[1];//Points *into* 'v'
v.push_back(12);//Inserts value 12
std::cout « *vptr;//Bug (possible use-after-free)
```

Bug?

`push_back` can allocate a new vector (if `v` full)
then, the reference `vptr` points to an invalid memory location
(it's a "dangling pointer") – `*vptr` might go wrong

We're humans: "42 little bugs in my code; 42 little bugs;
take one down, patch it around...

A more elaborate example, in C++

Iterator invalidator

```
std::vector<int> v { 10, 11 };//Declares&initialises
int *vptr = &v[1];//Points *into* 'v'
v.push_back(12);//Inserts value 12
std::cout « *vptr;//Bug (possible use-after-free)
```

Bug?

`push_back` can allocate a new vector (if `v` full)
then, the reference `vptr` points to an invalid memory location
(it's a "dangling pointer") – `*vptr` might go wrong

We're humans: "42 little bugs in my code; 42 little bugs;
take one down, patch it around... 63 little bugs in my code!"

The problem, in a nutshell

- Systems programming means writing code to run "close to the metal" (memory needs to be handled directly)
- Languages like C/C++ offer little help in this regard
- References are key for performance
  But aliasing leads to memory (and data-race) errors

Reasoning is hard but feasible
https://www.hackster.io/wallarug/c-memory-leaks-and-errors-with-examples-dda71f

What do we need?

Precise aliasing control mechanisms!

Some say "don't use these (C/C++) languages"

- `https://fossbytes.com/security-problem-with-c-c-and-why-you-shouldnt-use-it/`
- "Security problems are just bugs"
  Linus Torvalds, 2017
- The White House Office of the National Cyber Director
  Future Software Should Be Memory Safe
- C++ Alliance takes aim at C++ memory safety

Memory-safety in Rust

Rust

Memory-safety:

- none of the previous problems can happen at runtime
- no crashes

Rust

Memory-safety:

none of the previous problems can happen at runtime

no crashes

How's that possible?

Guaranteed at compile time, via a type system.

Rust

Memory-safety:

none of the previous problems can happen at runtime

no crashes

How's that possible?

Guaranteed at compile time, via a type system.

How does it work in Rust?

Explicit memory-management, but with precise aliasing control

Rust

Memory-safety:

none of the previous problems can happen at runtime

no crashes

How's that possible?

Guaranteed at compile time, via a type system.

How does it work in Rust?

Explicit memory-management, but with precise aliasing control

Resources

Official page `https://www.rust-lang.org/`

Presentation videos `https://research.mozilla.org/rust/`

Book `https://doc.rust-lang.org/book/`

Goal of Rust: memory and race safety, at compile time

This lecture

How this is achieved on single-threaded programs

memory safety =

forced initialisation + restricted aliasing + ownership

Goal of Rust: memory and race safety, at compile time

This lecture

How this is achieved on single-threaded programs

memory safety =

forced initialisation + restricted aliasing + ownership

Next lecture

How Rust deals with concurrency

Goal of Rust: memory and race safety, at compile time

This lecture

How this is achieved on single-threaded programs

memory safety =

forced initialisation + restricted aliasing + ownership

Next lecture

How Rust deals with concurrency – impossible with safe Rust!

Dealing with aliasing via ownership and borrowing

Restrict aliasing, as it is hard and dangerous

A firm approach

References are (by default) *unique* and *mutable*

Restrict aliasing, as it is hard and dangerous

A firm approach

References are (by default) *unique* and *mutable*

Ownership via *linear* types:

- each value has a single owner (reference is not aliased)
- owner might give away or lend their values
- when owner goes out of scope, the value is dropped

Restrict aliasing, as it is hard and dangerous

A firm approach

References are (by default) *unique* and *mutable*

Ownership via *linear* types:

- each value has a single owner (reference is not aliased)
- owner might give away or lend their values
- when owner goes out of scope, the value is dropped

When shared, values become *immutable*

Restrict aliasing, as it is hard and dangerous

A firm approach

References are (by default) *unique* and *mutable*

Ownership via *linear* types:

- each value has a single owner (reference is not aliased)
- owner might give away or lend their values
- when owner goes out of scope, the value is dropped

When shared, values become *immutable*

Rust type system tracks aliasing by "borrowing"references

Restrict aliasing, as it is hard and dangerous

A firm approach

References are (by default) *unique* and *mutable*

Ownership via *linear* types:

- each value has a single owner (reference is not aliased)
- owner might give away or lend their values
- when owner goes out of scope, the value is dropped

When shared, values become *immutable*

Rust type system tracks aliasing by "borrowing"references

- controlling them until found unusable (thus released)
- restoring then uniqueness and mutability

Restrict aliasing, as it is hard and dangerous

A firm approach

References are (by default) *unique* and *mutable*

Ownership via *linear* types:

- each value has a single owner (reference is not aliased)
- owner might give away or lend their values
- when owner goes out of scope, the value is dropped

When shared, values become *immutable*

Rust type system tracks aliasing by "borrowing"references

- controlling them until found unusable (thus released)
- restoring then uniqueness and mutability

We'll follow now "Diving Into Rust For The First Time"

`https://www.youtube.com/watch?v=_jMSrMex6R0`

Ownership and borrowing: three scenarios

Type	Ownership	Alias	Mutate
T	owned	X	OK
&T	shared	OK	X
&**mut** T	mutable	X	OK

Ownership and borrowing: three scenarios

Type	Ownership	Alias	Mutate
T	owned	X	OK
&T	shared	OK	X
&**mut** T	mutable	X	OK

if a reference is aliased,

- either all aliases are immutable, or
- there is only one, mutable

Ownership and borrowing: three scenarios

Type	Ownership	Alias	Mutate
T	owned	X	OK
&T	shared	OK	X
&**mut** T	mutable	X	OK

if a reference is aliased,

- either all aliases are immutable, or
- there is only one, mutable

when it's owned

safe full use (including mutation)

Ownership and borrowing: three scenarios

Type	Ownership	Alias	Mutate
T	owned	X	OK
&T	shared	OK	X
&**mut** T	mutable	X	OK

if a reference is aliased,

- either all aliases are immutable, or
- there is only one, mutable

when it's owned

safe full use (including mutation)

when borrowed, the owner cannot

- write or move it, if it's shared
- access it al all, if it's mutable

Owned mode (*linear* use)

A simple ownership example

```
fn main() {
    let mut s = String::from("hello");
    s.push_str(", world!");
    println!("{}", s);
}
```

A mutable, growable `String` requires allocating memory (unknown at compile time) on the heap:

- memory must be requested from the allocator at runtime (done by `String::from(...)`)
- memory must be returned to the allocator when `String::from(...)` is no longer used

Owned mode (*linear* use)

No aliasing – safe to mutate (exactly 1 owner)

- Ownership transfer makes the original owner lose the rights to the resource (it gave it away! Can't use anymore)

Owned mode (*linear* use)

No aliasing – safe to mutate (exactly 1 owner)

- Ownership transfer makes the original owner lose the rights to the resource (it gave it away! Can't use anymore)
- Once the reference is returned, full control is recovered

Owned mode (*linear* use)

No aliasing – safe to mutate (exactly 1 owner)

- Ownership transfer makes the original owner lose the rights to the resource (it gave it away! Can't use anymore)
- Once the reference is returned, full control is recovered
- When the owner drops a reference (moves out of its scope), the reference is destroid (GCed)

Owned mode (*linear* use)

No aliasing – safe to mutate (exactly 1 owner)

- Ownership transfer makes the original owner lose the rights to the resource (it gave it away! Can't use anymore)
- Once the reference is returned, full control is recovered
- When the owner drops a reference (moves out of its scope), the reference is destroid (GCed)

A simple example

```
let s1 = String::from("hello");
let s2 = s1;

println!(", world", s1);
```

Borrowing in action

```rust
let s1 = String::from("hello");
let s2 = s1;

println!(", world", s1);
```

```
error[E0382]: borrow of moved value: `s1`
 --> src/main.rs:5:28
  |
2 |     let s1 = String::from("hello");
  |         -- move occurs because `s1` has type `String`, which does not implement the `Copy` trait
3 |     let s2 = s1;
  |              -- value moved here
4 |
5 |     println!("{}, world!", s1);
  |                            ^^ value borrowed here after move
  |
```

More ownership examples

```
fn main() {
    let mut book = Vec::new(); // creates inst. of type Vec
```

More ownership examples

```
fn main() {
    let mut book = Vec::new(); // creates inst. of type Vec
    book.push(...);
```

More ownership examples

```
fn main() {
    let mut book = Vec::new(); // creates inst. of type Vec
    book.push(...);
    sell(book);
```

More ownership examples

```
fn main() {
    let mut book = Vec::new(); // creates inst. of type Vec
    book.push(...);
    sell(book);
    sell(book);
```

More ownership examples

```
fn main() {
    let mut book = Vec::new(); // creates inst. of type Vec
    book.push(...);
    sell(book);
    sell(book); // Error: use of moved value: 'book'
}
fn sell(book:Vec<String>) { ...}
```

More ownership examples

```
fn main() {
    let mut book = Vec::new(); // creates inst. of type Vec
    book.push(...);
    sell(book);
    sell(book); // Error: use of moved value: 'book'
}
fn sell(book:Vec<String>) { ...}
fn main() {
    let mut book = Vec::new();
    book.push(...);
    sell(book.clone());// passes a (deep) copy (of memory)
    sell(book);
}
fn sell(book:Vec<String>) { ...}
```

Passing a value to a fuction works like assignment

```rust
fn main() {
    let s = String::from("hello");  // s comes into scope

    takes_ownership(s);             // s's value moves into the function...
                                    // ... and so is no longer valid here

    let x = 5;                      // x comes into scope

    makes_copy(x);                  // x would move into the function,
                                    // but i32 is Copy, so it's okay to still
                                    // use x afterward

} // Here, x goes out of scope, then s. But because s's value was moved, nothing
  // special happens.

fn takes_ownership(some_string: String) { // some_string comes into scope
    println!("{}", some_string);
} // Here, some_string goes out of scope and `drop` is called. The backing
  // memory is freed.

fn makes_copy(some_integer: i32) { // some_integer comes into scope
    println!("{}", some_integer);
} // Here, some_integer goes out of scope. Nothing special happens.
```

- When the loan expires, the owner recovers full rights
- Lifetime: how long is a reference valid; allows to track borrows across functions (scope delimiters)

Returning values can also transfer ownership

```rust
fn main() {
    let s1 = gives_ownership();         // gives_ownership moves its return
                                        // value into s1
    let s2 = String::from("hello");     // s2 comes into scope
    let s3 = takes_and_gives_back(s2);  // s2 is moved into
                                        // takes_and_gives_back, which also
                                        // moves its return value into s3
} // Here, s3 goes out of scope and is dropped. s2 was moved, so nothing
  // happens. s1 goes out of scope and is dropped.

fn gives_ownership() -> String {             // gives_ownership will move its
                                             // return value into the function
                                             // that calls it

    let some_string = String::from("yours"); // some_string comes into scope

    some_string                              // some_string is returned and
                                             // moves out to the calling
                                             // function
}
// This function takes a String and returns one
fn takes_and_gives_back(a_string: String) -> String { // a_string comes into
                                                      // scope
    a_string  // a_string is returned and moves out to the calling function
}
```

Borrowing by reference

One can also create references to values and pass them to functions

```
fn main() {
    let s1 = String::from("hello");

    let len = calculate_length(&s1);

    println!("The length of '{}' is {}.", s1, len);
}

fn calculate_length(s: &String) -> usize {
    s.len()
}
```

Borrowing by reference

One can also create references to values and pass them to functions

```
fn main() {
    let s1 = String::from("hello");

    let len = calculate_length(&s1);

    println!("The length of '{}' is {}.", s1, len);
}

fn calculate_length(s: &String) -> usize {
    s.len()
}
```

References refer to values without owning them – dropping a reference does not drop its referred value

Borrowing by reference

One can also create references to values and pass them to functions

```
fn main() {
    let s1 = String::from("hello");
    change(&s1);
}

fn change(s: &String) {
    s.push_str(“, world”);
}
```

Borrowing by reference

One can also create references to values and pass them to functions

```
fn main() {
    let s1 = String::from("hello");
    change(&s1);
}

fn change(s: &String) {
    s.push_str(“, world”);
}
```

As variables are immutable by default, so are references

We’re not allowed to modify something we have a reference to

Borrowing by reference

One can also create mutable references

```rust
fn main() {
    let mut s = String::from("hello");

    change(&mut s);
}

fn change(some_string: &mut String) {
    some_string.push_str(", world");
}
```

Borrowing by reference

One can also create mutable references

```rust
fn main() {
    let mut s = String::from("hello");

    change(&mut s);
}

fn change(some_string: &mut String) {
    some_string.push_str(", world");
}
```

The type of change now states that it will mutate the value it borrows, but...

Borrowing by reference

One can also create mutable references

```rust
let mut s = String::from("hello");

let r1 = &mut s;
let r2 = &mut s;

println!("{}, {}", r1, r2);
```

Borrowing by reference

One can also create mutable references

```rust
let mut s = String::from("hello");

let r1 = &mut s;
let r2 = &mut s;

println!("{}, {}", r1, r2);
```

One cannot borrow a value as mutable more than once at a time!

Aliasing XOR Mutation

Rust *does not allow* multiple mutable references to the same data at the same time!

Immutable borrows CANNOT be combine with mutable ones

```rust
let mut s = String::from("hello");

let r1 = &s; // no problem
let r2 = &s; // no problem
let r3 = &mut s; // BIG PROBLEM

println!("{}, {}, and {}", r1, r2, r3);
```

Aliasing XOR Mutation

```rust
let mut s = String::from("hello");

let r1 = &s; // no problem
let r2 = &s; // no problem
println!("{} and {}", r1, r2);
// variables r1 and r2 will not be used after this point

let r3 = &mut s; // no problem
println!("{}", r3);
```

Dangling pointers are ill-typed

```rust
fn main() {
    let reference_to_nothing = dangle();
}

fn dangle() -> &String {
    let s = String::from("hello");

    &s
}
```

When `dangle()` returns, `s` is dropped (because `dangle()` owns `s`) so there is nothing to borrow...

Dangling pointers are ill-typed

```rust
fn no_dangle() -> String {
    let s = String::from("hello");

    s
}
```

`no_dangle()` no longer has a dangling pointer – ownership of s is moved out, so nothing is deallocated

Shared mode (*borrowed* use)

Multiple aliases – only read is allowed (no mutation!)

```
fn main() {
    let mut book = Vec::new();
    book.push(...);
    sell(&book);// passes by-reference – borrow the book
    sell(&book);
}
fn sell(book:&Vec<String>) { ... }
```

Shared mode (*borrowed* use)

Multiple aliases – only read is allowed (no mutation!)

```
fn main() {
    let mut book = Vec::new();
    book.push(...);
    sell(&book);// passes by-reference – borrow the book
    sell(&book);
}
fn sell(book:&Vec<String>) { ... } // no memory is freed
```

Shared borrow, example 2

```
let mut book = Vec::new();
book.push(...); // book is mutable
```

Shared borrow, example 2

```
let mut book = Vec::new();
book.push(...); // book is mutable
{ // scope for var ref
    let ref = &book; // book borrowed
```

Shared borrow, example 2

```
let mut book = Vec::new();
book.push(...); // book is mutable
{ // scope for var ref
    let ref = &book; // book borrowed
    ref.push(...);
```

Shared borrow, example 2

```
let mut book = Vec::new();
book.push(...); // book is mutable
{ // scope for var ref
    let ref = &book; // book borrowed
    ref.push(...); // Error: cannot mutate while shared
} // borrow ends
book.push(...);
```

Shared borrow, more examples

A variation of the previous example

```
let mut book = Vec::new();
book.push(...); // book is mutable
{
    let ref = &book; // book borrowed
    book.push(...);
```

Shared borrow, more examples

A variation of the previous example

```
let mut book = Vec::new();
book.push(...); // book is mutable
{
    let ref = &book; // book borrowed
    book.push(...); // Error: cannot mutate while shared
}
book.push(...);
```

Shared borrow, more examples

A variation of the previous example

```
let mut book = Vec::new();
book.push(...); // book is mutable
{
    let ref = &book; // book borrowed
    book.push(...); // Error: cannot mutate while shared
}
book.push(...);
```

Another example

```
let mut vect = Vec::new();
let elem = &vect[0]; //the whole vector borrowed here
```

Shared borrow, more examples

A variation of the previous example

```
let mut book = Vec::new();
book.push(...); // book is mutable
{
    let ref = &book; // book borrowed
    book.push(...); // Error: cannot mutate while shared
}
book.push(...);
```

Another example

```
let mut vect = Vec::new();
let elem = &vect[0]; //the whole vector borrowed here
vect.push(...);
```

Shared borrow, more examples

A variation of the previous example

```
let mut book = Vec::new();
book.push(...); // book is mutable
{
    let ref = &book; // book borrowed
    book.push(...); // Error: cannot mutate while shared
}
book.push(...);
```

Another example

```
let mut vect = Vec::new();
let elem = &vect[0]; //the whole vector borrowed here
vect.push(...); // Error: cannot mutate while shared
```

Mutable borrow mode (*linear* use)

Forbids using a reference as shared

- Disallows borrow as shared and mutable simultaneously

Mutable borrow mode (*linear* use)

Forbids using a reference as shared

- Disallows borrow as shared and mutable simultaneously

```
fn main() {
    let mut book = Vec::new();
    book.push(...); // book mutable
    revise(&mut book); // mutable borrow the book – locks it
    revise(&mut book); // retains ownership
}
fn revise(book:&Vec<String>) { ... } // mutable ref to book
```

Mutable borrow mode (*linear* use)

The variation, again: mutate in turns

```
let mut book = Vec::new();
book.push(...); // book mutable
{
    let ref = &mut book; // book borrowed
    ref.push(...);
```

Mutable borrow mode (*linear* use)

The variation, again: mutate in turns

```
let mut book = Vec::new();
book.push(...); // book mutable
{
    let ref = &mut book; // book borrowed
    ref.push(...);
    book.len(...);
```

Mutable borrow mode (*linear* use)

The variation, again: mutate in turns

```
let mut book = Vec::new();
book.push(...); // book mutable
{
    let ref = &mut book; // book borrowed
    ref.push(...);
    book.len(...); // Error: cannot access while shared
}
book.push(...); // ownership returned (borrow ended)
```

Lifetime of ref

Only the scope where's declared and used
(book borrowed by a (well de)limited period)

Rust allows to declare explicitly the lifetime of a reference